Numerical methods for thermal design

Which method to reach for, and how to tell when it has lied to you.

> **Why this guide exists.** In Modelica you wrote the equations and the compiler decided how to solve them. In Python you decide. That is more work and more control, and it means you have to know which method to reach for — and, more importantly, how to tell when one has quietly lied to you.

1 Three kinds of problem

YOU HAVE	REACH FOR	APPEARS IN
One equation, one unknown	`brentq` with a bracket	Weeks 2, 3, 4, 5, 10
Several coupled equations	`fsolve` (Newton)	Weeks 3, 7, 13
Maximise or minimise something	`minimize_scalar`, bounded	Weeks 11, 13

2 Root finding: always bracket if you can

A **bracket** is two values with the function on opposite signs. If one exists, Brent's method cannot fail. That guarantee is worth a great deal and it costs you only the effort of thinking about physical limits.

```
m = brentq(lambda m: pump_head(m) - system_loss(m), 1e-6, m_runout)
```

The bracket above is not arbitrary: flow cannot be negative, and it cannot exceed the pump's runout. **The physics gives you the bracket.**

> **The bracket error is information, not an obstacle.** `f(a) and f(b) must have different signs` usually means the answer is not where you assumed. In Week 4 that message appeared because a thick-insulation case could not shed the heat flow I had bracketed for — the bracket was wrong because my mental model was wrong.

Without a bracket you get the secant method, which needs a decent guess and can wander off. Use it only when you genuinely cannot bound the answer.

3 Coupled systems: Newton, and what it needs

`fsolve` takes a function returning a **vector of residuals** — each one an equation rearranged to "something = 0" — and a starting guess of the same length.

```
def residual(v):
    Th, Tw, Tc = v[:N], v[N:2*N], v[2*N:]
    r = []
    for i in range(N):
        r.append(C*(th_in - Th[i]) - G*(Th[i] - Tw[i]))   # hot side
        ...
    return np.array(r)

sol = fsolve(residual, guess)
```

Newton converges **quadratically** near the answer: the number of correct digits roughly doubles each step. It is spectacular when it works and it is not guaranteed to work at all.

The guess matters more than the method

Week 7 guesses a linear temperature profile between the two inlets. Week 3 guesses an exponential water profile down the tower and gets the outlet air enthalpy from an *exact* energy balance. Neither is arbitrary: both are **cheap approximations of the answer**. That is what a good initial guess is.

4 Fixed points, and damping

Some problems are naturally circular: you need x to compute x. Week 13 is the clearest case — the working-fluid flow sets the temperature profile, which sets the pinch, which sets the flow.

Successive substitution just feeds the new value back:

```
m = g(m)     # repeat until it stops moving
```

It is trivial to write and it can oscillate or diverge. The standard fix is **under-relaxation**: take only part of the proposed step.

```
m_next = m + damp*(g(m) - m)      # damp < 1
```

> **Damping is insurance, not acceleration.** Week 13 measures this: undamped converges in 3 iterations, `damp = 0.2` takes 100, and over-relaxing at 1.5 takes 36. You pay iterations for stability, and you only pay willingly on a problem that will not converge without it.

5 Optimisation

For one variable on a known interval, use the **bounded** method:

```
res = minimize_scalar(lambda p: -cop(p), bounds=(75e5, 130e5), method="bounded")
```

Note the minus sign: these routines minimise, so maximising means minimising the negative.

> **Do not use a three-point bracket for a moving optimum.** Week 11 originally used `bracket=(80e5, 95e5, 120e5)` and it failed as soon as the ambient temperature moved the peak, because the middle point was no longer the lowest. The bounded method has no such requirement.

Is it a global optimum?

These routines find a **local** optimum. The honest way to check is to plot the objective across the whole feasible range and look. Week 11 and Week 13 both do this before trusting the optimiser, and you should too. A single number from an optimiser, with no curve behind it, is not evidence.

6 Discretisation: how fine is fine enough?

When you chop a device into N segments, N is a numerical choice and it must be justified. The method is always the same: **refine until the answer stops moving**, and report the study.

Week 7 does it properly and finds something worth knowing: the segments needed for 1% error **grow with NTU**, because a high-NTU profile curves harder near the inlet. Resolution has to track the gradient, not the length.

> **Refinement across a discontinuity does not converge smoothly.** Week 6's condenser has three zones, and as N changes the zone boundaries fall at different places inside a segment, so the answer wobbles by a few tenths of a percent instead of settling. The honest fix is to put a node *on* each boundary. Recognising this pattern saves you chasing a bug that is not there.

7 How to tell when the solver has lied to you

Numerical methods fail quietly. They return a number. These are the checks worth building into every model you write:

1. **Close an energy balance.** It should come out at machine zero, around 10^{-12} relative. Week 3 prints one; Week 12 prints one. A non-zero residual means the solve did not converge, and nothing downstream is meaningful.
2. **Push a parameter to a limit you know the answer to.** Week 3 raises the tower's `KaV` by orders of magnitude and checks the approach falls to a floor. An infinitely large tower must deliver water at the wet bulb and not one kelvin below it.
3. **Check monotonicity where physics demands it.** The Week 3 tower floor must never get *warmer* as you add air. When it did, that was a missing validity test, not a solver problem — and it announced itself precisely because someone checked the direction.
4. **Check the ordering of quantities you know are ordered.** Week 5 requires homogeneous > Rouhani > Zivi void fraction at low quality.
5. **Compare against the number in the brief.** Every notebook states what it should produce.

8 Degrees of freedom, counted before you code

Count unknowns and independent equations **first**. If they do not match, no solver will help you: too few equations and the answer is not unique; too many and there is no answer at all. Week 13 does this explicitly, and it is the reason that case has exactly one decision variable to optimise.

> **A solver that converges instantly is suspicious.** While building Week 13, successive substitution converged in one step — because the "loop" was algebraically explicit and there was no real coupling. A fixed point that solves immediately is usually not a fixed point.